

SIMATS ENGINEERING
CO2_ASSESSMENT TOOL2_ Scenario Based Assignment

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192511061
Name	Vidhushini.T.S

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Code of Conduct:

I Vidhushini.T.S (192511061) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario:** A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	10
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	12.5
Decision Making	Makes well-reasoned, logical decisions	Makes reasonable decisions with	Makes weak decisions with	No clear decisions made or rationale provided	10

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
	with strong rationale	some justification	limited justification		
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand	7.5
				Total	50

Question 1. AI Drones for Emergency Medicine Delivery

Scenario :

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. Roads/paths may suddenly become blocked, weather changes travel costs, and the drone has only partial real-time information.

(i) Greedy Best-First Search :

Greedy Best-First Search selects the next node based mainly on the heuristic value $h(n)$, which estimates the distance from the current location to the destination.

$$f(n)=h(n)$$

In this scenario, the drone would use straight-line distance as its heuristic. It would generally choose the path that appears closest to the medicine delivery location.

Example:

Suppose the drone has three possible routes:

- Route A: 5 km straight-line distance
- Route B: 7 km
- Route C: 10 km

Greedy Best-First Search will prefer Route A because it has the smallest heuristic value.

Strengths :

- Very fast in many situations.
- Requires less computation than A*.
- Useful when an urgent decision is required.
- Can quickly move toward the destination.
- Suitable when approximate solutions are acceptable.

Weaknesses :

- Does not consider the actual cost already travelled.
- May choose a route with dangerous weather or difficult terrain.
- Can get trapped by a misleading heuristic.
- Does not guarantee the optimal route.
- With blocked paths, it may repeatedly explore poor alternatives.

For emergency medicine delivery, these weaknesses are important because the shortest-looking path may not be the safest or fastest path.

(ii) A* Search :

A* combines the actual cost of reaching a location with the estimated remaining cost.

$$f(n)=g(n)+h(n)$$

Where:

- $g(n)$ = actual cost from the starting point to node n
- $h(n)$ = estimated cost from n to the destination
- $f(n)$ = total estimated cost

For the drone:

- $g(n)$ can represent flight time, energy consumption, weather cost, or risk.
- $h(n)$ can represent straight-line distance to the destination.

For example:

Route	Actual cost $g(n)$	Heuristic $h(n)$	$f(n)$
A	15	5	20
B	10	8	18
C	8	15	23

A* would prefer Route B, because its total estimated cost is lowest.

(iii) Impact of Heuristic Accuracy :

The quality of the heuristic strongly affects both algorithms.

Greedy Best-First Search:

- Depends almost completely on $h(n)$.
- Accurate heuristic $\rightarrow$ faster and better decisions.
- Inaccurate heuristic $\rightarrow$ may select dangerous or inefficient paths.

A* :

- Uses both $g(n)$ and $h(n)$.
- Therefore, it is generally less sensitive to heuristic errors than Greedy Best-First Search.
- With an admissible heuristic that does not overestimate the true remaining cost, A* can guarantee an optimal solution in a static problem.

(iv) Effect of Dynamic Blocked Paths :

Floods can suddenly block a route that was previously available.

When this happens:

- The current search information becomes outdated.
- The drone must update its map.
- Previously generated paths may become invalid.
- The search algorithm may need to re-plan.

Greedy Best-First Search may quickly switch to another apparently close route, but that route may also be risky.

A* can reconsider the path using the updated $g(n)$ and $h(n)$ values. However, repeatedly running A* from scratch can be computationally expensive.

For highly dynamic environments, *incremental/re-planning algorithms such as D Lite** can be more appropriate.

(v) Recommended Algorithm :

*A is the better basic choice** because:

- It considers both actual travel cost and estimated remaining cost.
- It can incorporate weather, terrain and risk into the cost function.
- It generally provides better-quality routes than Greedy Best-First Search.
- It provides optimality guarantees under appropriate static assumptions.

However, because this is a real-time emergency application with frequently blocked paths, an incremental search method such as *D Lite**, based on A*-type principles would be even more suitable.

Use *A/D Lite* with real-time replanning and risk-aware cost functions.

2. Smart City Traffic Signal Optimization

Scenario :

A smart city wants to optimize traffic signals across hundreds of intersections. The objectives are to minimize waiting time, fuel consumption and congestion.

(i) Formulating the Problem as an Optimization Problem :

The problem can be represented as:

Minimize, $F = w_1W + w_2F_c + w_3C$

Where:

- W = total vehicle waiting time
- F_c = fuel consumption
- C = traffic congestion
- w_1, w_2, w_3 = importance weights

Decision Variables :

The system can control:

- Green-light duration
- Red-light duration
- Signal phase sequence
- Offset between nearby signals
- Cycle length

Objective :

Find signal settings that minimize overall traffic cost while satisfying constraints such as:

- Minimum/maximum green time
- Pedestrian crossing requirements
- Safety intervals
- Emergency vehicle priority

Why Traditional Search Is Inefficient

There may be hundreds of intersections, each with multiple possible signal timings.

If every intersection has several possible configurations, the number of possible combinations becomes enormous.

For example, if 100 intersections each have 10 possible configurations:

10^{100}

possible combinations exist.

Therefore, exhaustive search becomes impractical.

(ii) Hill Climbing :

Hill Climbing is a local search algorithm.

It starts with an initial solution and continuously moves to a neighboring solution that improves the objective.

Example:

Initial signal timing:

Green = 40 seconds

The algorithm tests:

- 45 seconds → better traffic flow
- 50 seconds → better
- 55 seconds → better
- 60 seconds → worse

It stops at 55 seconds because no immediate neighboring solution is better.

Advantages :

- Simple to implement.
- Requires relatively little memory.
- Fast for large problems.
- Useful for continuously improving an existing solution.

Limitations :

The major problem is that Hill Climbing only looks at nearby solutions.

It may therefore stop at a solution that is locally good but globally poor.

(iii) Local Maxima, Plateaus and Ridges :

Local Maximum

A local maximum is a solution that is better than its immediate neighbors but not the best possible solution.

Traffic example:

Changing one signal improves traffic, but reaching a much better city-wide solution requires temporarily making that signal worse.

Hill Climbing refuses to make that temporary move and gets stuck.

Plateau

A plateau occurs when many neighboring solutions have almost the same evaluation value.

Example:

Changing a signal from 40 to 41, 42 or 43 seconds produces almost the same traffic performance.

The algorithm cannot identify a clear direction for improvement.

Ridge

A ridge is a narrow region where the best direction requires a sequence of coordinated changes.

Example:

Two connected intersections need their green times changed together. Changing only one intersection makes traffic worse, so Hill Climbing cannot reach the better solution.

(iv) Simulated Annealing :

Simulated Annealing allows the algorithm to sometimes accept a worse solution.

Initially, the algorithm has a high temperature, so it can explore many possibilities.

As the temperature decreases, it becomes more selective.

This helps it escape local optima.

Traffic Example

Suppose:

- Current solution = good
- New solution = temporarily worse
- Simulated Annealing may accept the worse solution.

This can lead to another region of the search space where a much better solution exists.

(v) Genetic Algorithms :

A Genetic Algorithm treats possible traffic configurations as a population of solutions.

Basic steps:

1. Generate initial population.
2. Evaluate each traffic configuration.
3. Select better solutions.
4. Perform crossover.
5. Perform mutation.
6. Create a new generation.
7. Repeat until a good solution is found.

For example, one solution may contain:

[40, 50, 35, 60, ...]

representing green-light durations at different intersections.

Genetic Algorithms are particularly useful when many variables interact.

(vi) Recommended Approach :

For a large-scale smart city, a Genetic Algorithm or hybrid Genetic Algorithm + local search would be highly suitable.

A GA can:

- Search for a huge solution space.
- Avoid getting easily trapped in local optima.
- Handle many interacting intersections.
- Adapt to changing traffic patterns.

For real-time control, however, continuously running a large GA may be expensive. Therefore, a practical architecture would use:

GA → generate good signal plans

Local/fast optimization → continuously adjust them

Real-time traffic data → update the optimization

A hybrid Genetic Algorithm + local search/receding-horizon optimization is recommended because it provides a good balance between scalability, adaptability and solution quality.

3. Autonomous Mars Rover

Scenario :

An autonomous rover explores unknown Martian terrain. It must avoid hazards, collect samples and make decisions without having a complete map.

(i) Why an Online Search Agent Is Required :

An offline search agent assumes that the environment and problem are known before the search begins.

For example:

Start $\rightarrow$ A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ Goal

If the entire map is known, the rover can calculate a path before moving.

However, the Mars rover does not have complete information.

It may discover:

- A crater that was previously unknown.
- Rocks blocking the route.
- Dangerous terrain.
- New areas containing samples.

Therefore, it must act and learn simultaneously.

This is the purpose of an online search agent.

Online Search

The rover:

1. Senses the environment.
2. Updates its knowledge.
3. Selects an action.
4. Moves.
5. Receives new information.
6. Updates its model again.
7. Replans.

This creates a continuous sense $\rightarrow$ plan $\rightarrow$ act $\rightarrow$ learn cycle.

(ii) Partial Observability and Dynamic Environment :

Partial Observability

The rover has limited sensors.

It cannot see the complete environment from its current location.

For example:

The rover sees clear terrain ahead, but a crater may exist behind a nearby rock.

Therefore, its decisions are based on incomplete information.

Dynamic Environment

Although Mars changes more slowly than a city, the rover's operational environment can change because of:

- Dust storms
- Sensor uncertainty
- Changing lighting
- Wheel slip
- Unexpected terrain
- Hardware limitations

Therefore, the rover must continuously update its understanding.

(iii) Internal Model :

The rover maintains an internal representation of its environment.

It can store:

- Known safe locations
- Known obstacles
- Unknown locations
- Terrain costs
- Previously visited locations
- Sample locations
- Estimated distances

As new sensor data arrives, the model is updated.

Example:

Before sensing:

$A \rightarrow B \rightarrow C$

The rover believes B is safe.

After sensing:

$A \rightarrow B = \text{CRATER}$

The rover updates its model:

$A \rightarrow D \rightarrow C$

and searches for a new route.

(iv) Online Search vs Uninformed and Informed Search :

Feature	Uninformed Search	Informed Search	Online Search
Environment knowledge	Usually known	Usually known	Partially known
Uses heuristic	No	Yes	Can use one
Complete map required	Usually	Usually	No
Replanning	Limited	Possible	Essential
Suitable for rover	Poor	Better	Excellent

Uninformed Search

Examples:

- BFS
- DFS
- Uniform Cost Search

These do not use additional knowledge about the destination.

Informed Search

Examples:

- Greedy Best-First Search
- A*

These use heuristic information to guide the search.

Online Search

Online search is particularly suitable when the environment is unknown and the agent must discover information through actions.

(v) Recommended Approach :

The best approach is an online, heuristic-guided search with continuous replanning.

A practical rover can use:

- Sensor-based environment mapping
- A*/D* Lite-type path planning
- Continuous obstacle detection
- Risk-aware cost functions
- Replanning when new information is discovered

This provides:

- Adaptability
- Safety
- Efficient exploration
- Response to unknown terrain

An online informed search approach with incremental replanning is most suitable because the rover cannot depend on a complete map and must continuously adapt to newly discovered hazards.

4. University Exam Timetable Using CSP

Scenario :

A university wants to automatically generate exam timetables while satisfying student, hall, faculty, prerequisite and accommodation constraints.

(i) Formulation as a CSP :

A Constraint Satisfaction Problem (CSP) consists of:

$CSP = (X, D, C)$

Where:

- X = Variables
- D = Domains
- C = Constraints

(ii) Variables :

Each exam can be represented as a variable.

For example:

- X_1 = Mathematics
- X_2 = Data Structures
- X_3 = DBMS
- X_4 = Artificial Intelligence

The value assigned to each variable represents its:

- Date
- Time slot
- Examination hall

For example:

X_1 = Monday, 9 AM, Hall 1

(iii) Domains :

The domain contains possible values for each variable.

Example:

Mathematics:

- Monday 9 AM, Hall 1

- Monday 9 AM, Hall 2
- Monday 2 PM, Hall 1
- Tuesday 9 AM, Hall 1

The system eliminates values that violate constraints.

(iv) Constraints :

1. No Student Exam Overlap

If students take both Mathematics and DBMS:

$\text{Time}(\text{Maths}) \neq \text{Time}(\text{DBMS})$

2. Hall Capacity

The number of students assigned to a hall must not exceed its capacity.

$\text{Students} \leq \text{HallCapacity}$

3. Prerequisite Constraints

If Subject A must occur before Subject B:

$\text{Time}(\text{A}) < \text{Time}(\text{B})$

4. Faculty Availability

An exam cannot be scheduled when the required faculty member is unavailable.

5. Special Accommodation

Students requiring special facilities must be assigned suitable halls.

6. Hall Conflict

One hall cannot conduct two exams at the same time.

(v) Backtracking Search :

Backtracking assigns values to variables one at a time.

Example:

1. Schedule Mathematics.
2. Schedule DBMS.
3. Schedule AI.
4. Check constraints.

5. If a conflict occurs, undo the last assignment.
6. Try another available value.
7. Continue until a complete valid timetable is found.

Example:

Maths → Monday 9 AM
DBMS → Monday 9 AM

If students have both exams, this creates a conflict.

The algorithm backtracks:

Maths → Monday 9 AM
DBMS → Monday 2 PM

Now the constraint may be satisfied.

(vi) Constraint Propagation / Forward Checking :

Forward checking improves backtracking by removing invalid choices from the domains of unassigned variables.

Example:

Suppose:

Maths = Monday 9 AM

and many students take Maths and DBMS.

The system immediately removes:

Monday 9 AM

from the possible DBMS slots.

Instead of waiting until DBMS is assigned, the conflict is detected early.

Benefits

- Reduces unnecessary search.
- Detects conflicts earlier.
- Reduces the search space.
- Makes timetable generation faster.

Other useful CSP techniques include:

- MRV — Minimum Remaining Values
- Degree heuristic

- Least Constraining Value
- Arc consistency

(vii) Real-World Challenges :

The university may have:

- Hundreds of courses
- Thousands of students
- Limited halls
- Faculty availability
- Multiple departments
- Special accommodations
- Different exam durations
- Holiday restrictions
- Last-minute changes

As the number of variables increases, the search space grows rapidly.

Best Strategy

Use:

Backtracking + Forward Checking + Constraint Propagation + Heuristics

For very large universities, this can be combined with optimization techniques such as local search or specialized CSP/constraint-programming solvers.

A constraint-based approach using backtracking with forward checking and MRV is highly suitable because it satisfies hard constraints while reducing unnecessary search and improving scalability.

5. Strategic Game AI — Minimax and Alpha-Beta Pruning

Scenario :

A gaming company is developing an AI opponent for a real-time strategy game. The game has a very large decision tree and strict time limits.

(i) Minimax Algorithm :

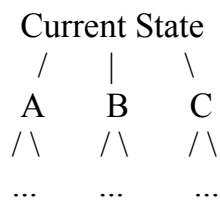
Minimax is designed for two-player, adversarial games.

It assumes:

- One player tries to maximize the score.
- The opponent tries to minimize the score.

The AI represents the game as a tree.

Example:



The AI explores possible moves and predicts the opponent's responses.

MAX Level

The AI chooses the move with the highest evaluation.

MIN Level

The opponent is assumed to choose the move that produces the lowest evaluation for the AI.

Therefore:

$$\text{Minimax}(s) = \begin{cases} \text{Utility}(s), & \text{if terminal state} \\ \text{Max}(\text{Minimax}(\text{children})), & \text{MAX} \\ \text{Min}(\text{Minimax}(\text{children})), & \text{MIN} \end{cases}$$

(ii) Computational Challenges :

The major problem is the enormous size of the game tree.

If:

- b = branching factor
- d = search depth

then the approximate time complexity is:

$$O(b^d)$$

For example, if a game has:

$$b = 10$$

and the AI searches:

$$d = 8$$

then:

$$10^8 = 100,000,000$$

possible nodes may need to be considered.

This is too expensive for strict real-time decision making.

(iii) Alpha-Beta Pruning :

Alpha-Beta pruning improves Minimax by eliminating branches that cannot affect the final decision.

It maintains two values:

Alpha (α)

The best value that MAX can guarantee so far.

Beta (β)

The best value that MIN can guarantee so far.

When:

$$\alpha \geq \beta$$

The remaining branch can be pruned because it cannot change the final decision.

(iv) Detailed Example :

Suppose MAX is evaluating two moves:



First, A is evaluated and gives:

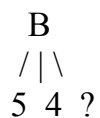
$$A = 8$$

So MAX currently has:

$$\alpha = 8 \quad \alpha = 8$$

Now MAX evaluates B.

Suppose B leads to a MIN node:



MIN will choose the lowest value.

After seeing 5 and 4:

$$\beta = 4$$

Since:

$$\beta(4) < \alpha(8)$$

MAX already has an alternative producing 8.

MIN can force B to be at most 4, so the remaining child of B does not need to be evaluated.

That child is pruned.

Result

Instead of evaluating every possible move, Alpha-Beta evaluates only branches that can affect the final decision.

With excellent move ordering, its effective best-case complexity can approach:

$$O(b^{d/2})$$

which allows much deeper search than ordinary Minimax.

(v) Designing the Evaluation Function :

Real-time games cannot search until the final game state because the tree is too large.

Therefore, the AI stops at a fixed depth and evaluates the current state.

A possible evaluation function is:

$$\text{Evaluation}(s) = w_1M + w_2H + w_3R + w_4E + w_5P$$

Where:

- M = material/unit strength
- H = health of important units
- R = resource availability
- E = territory/control
- P = positional advantage
- $w_1 \dots w_5$ = importance weights

Factors to Consider :

1. Unit strength

More powerful surviving units should increase the score.

2. Health

Healthy units are more valuable than damaged units.

3. Resources

More resources can allow the player to build stronger forces.

4. Territory

Controlling important areas can provide strategic advantages.

5. Position

Units in strategically strong positions receive higher scores.

6. Threat level

Immediate threats from the opponent should reduce the evaluation.

(vi) Recommended Strategy for Real-Time Performance :

Pure Minimax is not suitable for a very large real-time strategy game because the search tree is enormous.

A better approach is:

Depth-Limited Minimax + Alpha-Beta Pruning + Iterative Deepening

Depth-Limited Minimax

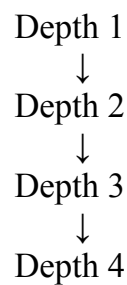
Search only up to a fixed depth.

Alpha-Beta Pruning

Remove branches that cannot influence the decision.

Iterative Deepening

Search progressively deeper:



If the time limit expires, the AI can use the best result obtained from the previous completed depth.

Additional Improvements :

- Good move ordering
- Transposition tables
- Caching previously evaluated states
- Time limits
- Domain-specific evaluation functions

Depth-limited Minimax with Alpha-Beta pruning and iterative deepening is the most suitable strategy because it provides a strong balance between decision quality, computational efficiency and strict real-time response requirements.